

TeachLink

An Intelligent Student Risk Detection and Performance Monitoring System

IMPLEMENTATION PLAN

Contents

1.0 Introduction.....	3
1.1 System at a Glance	3
2.0 Implementation Schedule	4
2.1 Phase Overview.....	4
2.2 Phase 1 — Project Setup and Database (Weeks 1–2).....	4
2.3 Phase 2 — Courses, Lessons, and Quizzes (Weeks 3–5)	5
2.4 Phase 3 — Risk Engine and Analytics (Weeks 6–8)	5
2.5 Phase 4 — Testing, Deployment, and Documentation (Weeks 9–10)	6
3.0 Installation and Conversion Plans.....	7
3.1 Software Installation Plan	7
3.2 Hardware Requirements.....	8
3.3 Conversion Activities	10
4.0 Training Plan	12
4.1 Training Objectives	12
4.2 Who Needs What Training	12
4.3 Training Methods	12
4.4 Training Schedule.....	14
4.5 Training Materials to Prepare	14
5.0 Software Maintenance Plan	15
5.1 Types of Maintenance That Will Be Needed	15
5.2 Who Is Responsible for What	15
5.3 Bug Reporting and Fix Process.....	15
5.4 Database Backup Procedure.....	16
5.5 Keeping Dependencies Up to Date	16
5.6 Maintenance Log	17
6.0 Change Management Plan.....	18
6.1 Categories of Change.....	18

6.2 The Change Request Process	18
6.3 Roles in the Change Process	19
6.4 Impact Assessment Checklist.....	19
6.5 Change Request Form	19
6.6 Telling Users About Changes.....	20
6.7 Rollback Plan	20
7.0 Conclusion	22

1.0 Introduction

This document is the Implementation Plan for TeachLink, my Final Year Project. TeachLink is a web application I built for independent educators to help them detect which students are at risk of falling behind or dropping out — before things get too bad to fix.

The idea came from a real problem. Independent tutors have no proper tools for this. They usually rely on WhatsApp messages and personal notes to track how students are doing, which means they often only realise a student is struggling once that student has already stopped engaging. TeachLink changes that by automatically calculating a risk score for every enrolled student based on three things: how far they have progressed through the course, how they are performing in quizzes, and how recently they logged in.

This implementation plan covers everything needed to take TeachLink from a working codebase to a fully deployed system that real users can operate. The plan is organised into six areas: the implementation schedule, installation and conversion, training, software maintenance, and change management.

1.1 System at a Glance

Item	Detail
System Name	TeachLink
What It Does	Monitors student progress, quiz scores, and engagement to flag at-risk learners automatically
Who Uses It	Independent educators (Teacher role) and their enrolled learners (Student role)
Backend	Python 3.10 with Django 4.2.7 web framework
Database	PostgreSQL 14 — stores all users, courses, quiz results, risk scores, and alert history
Frontend	Django templates with Bootstrap 5.3 for layout and Chart.js 4.4 for analytics charts
Risk Engine Location	analytics/engine.py — <code>recalculate_risk()</code> is called every time a student completes a lesson or submits a quiz

2.0 Implementation Schedule

I divided the project into four phases. I did not try to build everything at once — I started with the foundation (database and authentication), then added the course system, then the risk engine, and finally testing and documentation. This order made sense because the analytics engine depends on having real quiz results and lesson completion data to work with, so that had to come last.

2.1 Phase Overview

Phase	Weeks	What I Was Building	Main Outputs	Status
Phase 1	1 – 2	Project skeleton and database	Django project, PostgreSQL, custom User model, login/register pages	Complete
Phase 2	3 – 5	Courses, lessons, and quizzes	Full course builder, quiz grading, enrolment, lesson completion tracking	Complete
Phase 3	6 – 8	Risk engine and analytics dashboards	Risk scores, trends, alerts, difficulty analyser, teacher and student dashboards	Complete
Phase 4	9 – 10	Testing, deployment, and documentation	Unit tests, system testing, deployment configuration, final FYP docs	In Progress

2.2 Phase 1 — Project Setup and Database (Weeks 1–2)

The first thing I did was set up the Django project structure and connect it to PostgreSQL. I chose PostgreSQL over SQLite because I needed a database that could handle relational queries across multiple tables — for example, joining enrollments to risk scores to quiz results. Django's ORM handles most of this, but PostgreSQL performs better for the kinds of aggregation queries the analytics views use.

Task	App / File	Days	Status
Set up Django project folder structure (teachlink/, manage.py, wsgi.py)	Project root	1	Done
Configure PostgreSQL in settings.py — DATABASES block with host, port, name, user, password	teachlink/settings.py	0.5	Done
Build custom User model with a role field (Teacher / Student) — Django's built-in user does not support roles	accounts/models.py	1	Done
Write register, login, and logout views with role selection on registration	accounts/views.py	1	Done
Generate and apply initial migrations for all apps	All apps	0.5	Done
Configure INSTALLED_APPS, AUTH_USER_MODEL, and URL routing in settings and urls.py	teachlink/	0.5	Done

Build base.html — shared navbar, sidebar, Bootstrap 5 and Chart.js loaded from CDN	templates/base.html	1	Done
--	---------------------	---	------

2.3 Phase 2 — Courses, Lessons, and Quizzes (Weeks 3–5)

Phase 2 was the largest in terms of number of views. The course builder needed to support a three-level hierarchy: Course → Module → Lesson, with an optional Quiz attached to each Lesson. I also needed to track two different things per lesson per student: whether they completed it (LessonCompletion) and how many times they accessed it (LessonAccess). The access count is used later by the difficulty analyser.

Task	App / File	Days	Status
Build Course, Module, Lesson, Quiz, Question, Choice models	courses/models.py	2	Done
Teacher views: create, edit, delete course; add/remove modules and lessons	courses/views.py	1.5	Done
Quiz builder views: add questions, add choices, set pass score per quiz	courses/views.py	2	Done
Student views: browse published courses, enrol, view lessons	courses/views.py	1	Done
Auto-graded quiz submission — marks each answer, stores score in QuizResult	courses/views.py	1.5	Done
Lesson completion tracking — creates LessonCompletion record on first view, recalculates progress_percentage	courses/views.py	1	Done
LessonAccess tracking — records every time a student opens a lesson (used by difficulty analyser)	courses/models.py	0.5	Done

2.4 Phase 3 — Risk Engine and Analytics (Weeks 6–8)

This was the most technically involved phase and the core academic contribution of the project. The risk engine lives in analytics/engine.py and is called recalculate_risk(enrollment). It runs automatically every time a student does something — finishes a lesson, submits a quiz, or visits a course page.

The risk score formula is:

$$\text{risk_score} = (1 - \text{progress}\%) \times 0.40 + (1 - \text{avg_quiz_score}\%) \times 0.40 + (\text{days_inactive} / 14) \times 0.20$$

A score below 0.40 is Low, between 0.40 and 0.69 is Medium, and 0.70 or above is High. The trend is calculated by comparing the new score to the previous one — a drop of more than 0.05 counts as Improving, a rise of more than 0.05 is Declining.

Task	App / File	Days	Status
------	------------	------	--------

Write calculate_risk_score() and classify_risk() in engine.py	analytics/engine.py	1	Done
Write calculate_trend() — compares previous and current score	analytics/engine.py	0.5	Done
Write calculate_engagement() — lesson rate × 0.4 + quiz rate × 0.3 + login score × 0.3	analytics/engine.py	1	Done
Write update_lesson_difficulty() — failure_rate × 0.7 + access_penalty × 0.3	analytics/engine.py	1	Done
Write generate_alerts() — 3-level alert logic with deduplication (one alert per level per day)	analytics/engine.py	1.5	Done
Build RiskHistory model — saves a record each hour, used for the line chart	analytics/models.py	0.5	Done
Build teacher analytics dashboard — risk distribution donut chart, cohort stats, student table	analytics/views.py + templates	2	Done
Build student self-analytics page — progress vs class average, engagement trend	analytics/views.py + templates	1	Done
Build individual student detail page — 6-week risk history chart, alert history, intervention log	analytics/views.py + templates	1	Done

2.5 Phase 4 — Testing, Deployment, and Documentation (Weeks 9–10)

Phase 4 is currently in progress. The main outstanding tasks are writing the unit tests for the risk engine (which is pure Python, so it is easy to test without a database), running end-to-end tests through the full user flows, and finalising all documentation.

Task	Location	Days	Status
Write unit tests for risk engine: test score boundaries, trend logic, engagement formula	analytics/tests.py	1.5	In Progress
Full integration test — register teacher, create course, enrol student, submit quiz, check dashboard	Manual / test scripts	2	Planned
User acceptance testing using a test teacher account with sample data	Staging server	1	Planned
Fix any bugs found during testing	All apps	1–2	Planned
Set DEBUG=False, configure ALLOWED_HOSTS and STATIC_ROOT for production	teachlink/settings.py	0.5	Planned
Run python manage.py collectstatic and verify deployment on production server	Server	1	Planned
Finalise implementation plan, testing report, and all FYP documentation	Project docs	2	Planned

3.0 Installation and Conversion Plans

This section covers two things: how to set up TeachLink on a machine from scratch, and how to transition educators from their current informal setup onto the new system. I will start with the software requirements, then hardware, and then the conversion strategy.

3.1 Software Installation Plan

TeachLink does not require any installation on the teacher's or student's own computer — they just need a web browser. The only machine that needs software installed is the one running the server. In development, that is my laptop. In production, it would be a cloud server or a dedicated machine at the educator's organisation.

3.1.1 Server Software Requirements

All of the following must be installed on the server machine before TeachLink can run:

Package	Version	Why It Is Needed
Python	3.10 or above	Django runs on Python. Version 3.10 introduced better error messages and type hints that make debugging easier.
Django	4.2.7	The web framework. Handles URL routing, views, the ORM (database layer), session management, and the template engine.
PostgreSQL	14 or above	The relational database. All user accounts, courses, enrollments, quiz results, risk scores, and alert records are stored here.
psycopg2-binary	2.9.9	The Python driver that lets Django talk to PostgreSQL. Without this, Django cannot connect to the database at all.
django-crispy-forms	2.1	Renders Django forms with consistent, styled HTML. Used on the registration, login, and quiz builder forms.
crispy-bootstrap5	0.7	The Bootstrap 5 template pack for crispy-forms. Needed alongside django-crispy-forms.
Pillow	10.1.0	Python imaging library. Included as a dependency for media file handling. Not heavily used in the current version.

3.1.2 What End Users Need (Teachers and Students)

No installation is needed on the user's side. The only requirements are:

Requirement	Details
-------------	---------

Web browser	Any modern browser works — Chrome, Firefox, Edge, or Safari. Chrome is recommended for the dashboards since it renders Chart.js charts most consistently.
JavaScript enabled	The risk distribution chart and risk history line chart are rendered by Chart.js, which runs as JavaScript in the browser. If JavaScript is disabled, the charts will not appear.
Internet connection	Required at all times. TeachLink is a server-side application — every page load is a request to the server.
Screen size	The layout uses Bootstrap's responsive grid, so it works on mobile and tablet. However, the student risk table on the teacher dashboard is best viewed on a screen wider than 1200px.

3.1.3 Step-by-Step Installation

These steps assume a clean machine running Windows, Ubuntu, or macOS with no prior Python or PostgreSQL installation.

1. Install Python 3.10 or later from python.org. After installation, confirm it worked by opening a terminal and running: `python --version`. You should see something like Python 3.11.4.
2. Install PostgreSQL from postgres.org. During the installation wizard, set a password for the postgres superuser and make a note of it — you will need it in Step 7.
3. Copy the TeachLink project folder onto the machine. The folder contains `manage.py`, `requirements.txt`, and the app directories (`accounts`, `courses`, `analytics`, `core`).
4. Open a terminal and navigate into the project folder — the one containing `manage.py`.
5. Install all Python dependencies in one command: `pip install -r requirements.txt` — this installs Django, `psycopg2`, `crispy-forms`, and everything else the project needs.
6. Create a blank PostgreSQL database by running: `psql -U postgres -c "CREATE DATABASE teachlink_db;"` — enter the postgres password when prompted.
7. Open the file `teachlink/settings.py` in a text editor. Find the DATABASES section and fill in NAME (`teachlink_db`), USER (`postgres`), PASSWORD (your password from Step 2), HOST (`localhost`), and PORT (`5432`).
8. Generate migration files for all four apps: `python manage.py makemigrations accounts courses analytics core` — this creates the SQL schema instructions from the Python model definitions.
9. Apply the migrations to actually create the database tables: `python manage.py migrate`
10. Create the superuser (admin) account: `python manage.py createsuperuser` — enter a username, email address, and password.
11. Start the server: `python manage.py runserver` — the terminal will say 'Starting development server at `http://127.0.0.1:8000/`'.
12. Open a browser and go to `http://127.0.0.1:8000`. The TeachLink landing page should appear. If it does not, check the terminal for error messages.

3.2 Hardware Requirements

TeachLink is a lightweight application. It does not require dedicated server hardware — it can run comfortably on a standard laptop during development, and on a low-cost cloud VM in production. The table below gives minimum and recommended specs.

Component	Minimum	Recommended
Processor	Dual-core 1.8 GHz	Quad-core 2.5 GHz — helpful when PostgreSQL and Django are running concurrently
RAM	4 GB	8 GB — PostgreSQL caches frequently accessed tables in memory; more RAM means faster queries
Storage	20 GB free	50 GB SSD — SSD storage speeds up PostgreSQL read/write significantly compared to a spinning disk
Operating System	Windows 10, Ubuntu 20.04, or macOS 11	Ubuntu 22.04 LTS for a production server — better long-term support and Django deployment documentation
Network	Broadband, at least 5 Mbps	10 Mbps or above if multiple students will be using the system simultaneously

3.2.1 Hardware Setup Notes

1. Confirm the machine meets the minimum specs before installing PostgreSQL — the database server uses memory continuously while running.
2. Check that your operating system supports Python 3.10+ and the version of PostgreSQL you plan to install.
3. If the system will be used by students on different networks (i.e., not just locally), the server needs a public IP address. On a local network, the default Django server URL (127.0.0.1:8000) is only accessible from the same machine.
4. For a production deployment, the server needs ports 80 (HTTP) and 443 (HTTPS) open in the firewall. Port 5432 (PostgreSQL) should be kept closed externally.

3.3 Conversion Activities

Conversion here means the process of moving an educator from their existing way of managing students — typically a mix of WhatsApp, shared Google Sheets, and personal notes — to using TeachLink as their primary tool.

There is no automated data import in the current version. TeachLink does not have a CSV upload feature or a way to import historical grades. This is a deliberate decision: the system is designed for ongoing monitoring from the point a student first enrolls, not for analysing past records. Any historical data the educator wants to keep should stay in its original format (spreadsheet, document) and can be used as a reference alongside TeachLink.

3.3.1 Things to Do Before Going Live

1. Collect a list of all students the educator currently teaches — names, email addresses, and which course(s) they are enrolled in.
2. List all courses being taught and what topics/lessons each one contains. This will be used to recreate the course structure in TeachLink.
3. Gather any existing quiz or assessment questions. These will need to be entered manually into the quiz builder.
4. Back up all existing records in their current format. Once TeachLink is live, those old records are not deleted — they just become the historical baseline.
5. Agree on a go-live date with the educator. Avoid starting mid-course if possible — starting at the beginning of a new course intake gives the risk engine the most complete data to work with.

3.3.2 Setting Up the System with Real Data

Because there is no automated import, the teacher needs to manually recreate their course structure in TeachLink. The steps are:

1. The educator registers a Teacher account at </accounts/register/>, enters their display name, and logs in.
2. Using the Course Builder, the teacher creates each course — giving it a title and description. The course is saved as a Draft until it is ready to publish.
3. For each course, the teacher adds Modules (which are like chapters or topic groups) and then adds Lessons inside each module. Lesson content is typed directly into the lesson form.
4. If the educator uses quizzes, they open each lesson and click 'Add Quiz', then add multiple-choice questions one by one through the quiz builder. They set a pass mark (percentage) for each quiz.
5. Once a course is complete, the teacher publishes it using the toggle on the course settings page. It then appears in the student browse view.
6. Students register their own accounts, browse the available courses, and click Enrol. The teacher can see new enrolments on their dashboard immediately.

Important: The risk engine starts with default values (0% progress, 0 quiz score, 0 days inactive) for a brand-new enrolment. Risk scores will be meaningful after the first few lessons and quiz submissions. In the first few days, the dashboard will show mostly Low risk across the board — this is expected.

3.3.3 Conversion Strategy

I am recommending a Direct Cutover strategy for deploying TeachLink. This means the educator switches to TeachLink on a fixed date and stops using their old tools (WhatsApp tracking, spreadsheets) for student monitoring from that point forward.

The reason I chose Direct Cutover over a phased or parallel approach is straightforward: there is no legacy system to maintain in parallel. Independent educators are not running a formal LMS that needs careful migration — they are running informal processes. Running both in parallel would just confuse students about which platform to use and double the educator's workload.

The cutover will happen in this order:

1. Complete all course setup, quiz entry, and testing at least three days before the go-live date.
2. Do a final walkthrough of the system — log in as a test student, enrol in a course, complete a lesson, and submit a quiz — to confirm everything works end to end.
3. Communicate the go-live date to students at least five days in advance, with the URL and registration instructions.
4. On the go-live date: publish all courses, confirm students can access and enrol, and begin monitoring the dashboard.
5. Keep the old WhatsApp group or communication channel active for the first two weeks as a fallback if any student has trouble logging in or accessing the system.
6. After two weeks, close the fallback channels and operate exclusively through TeachLink.

4.0 Training Plan

Training for TeachLink needs to be simple and focused. The educator is not a software developer — they need to understand what the risk score means and what to do when an alert appears, not how the algorithm works underneath. Students need even less — just enough to register, find their course, and submit work.

I have designed separate training approaches for each user type because their tasks and technical comfort levels are different.

4.1 Training Objectives

By the end of training, each user should be able to do the following without help:

- Teachers: create a course with modules, lessons, and quizzes; read the analytics dashboard; understand what a risk level and alert level mean; and send an intervention message.
- Students: register, find and enrol in a course, complete a lesson, submit a quiz, and check their own analytics.
- Admin: log in to the Django admin panel, inspect records, and reset data during testing.

4.2 Who Needs What Training

User	Tasks They Need to Perform	Starting Knowledge Level
Teacher	Build course structure, add quizzes, monitor the analytics dashboard, interpret risk scores and engagement levels, respond to alerts by sending messages	Comfortable with web browsers and basic office software. No prior experience with a dedicated LMS required.
Student	Register with student role, browse published courses, enrol, view lesson content, submit quizzes, view own progress	Basic computer and smartphone literacy. Students can generally figure out the interface without much guidance.
Admin	Access /admin panel, inspect enrollment and risk history records, reset test data	Some technical background. Understands what a database table is.

4.3 Training Methods

4.3.1 Live Walkthrough — for Teachers

The teacher will go through the system with me (the developer) in a one-on-one session lasting about 2 to 3 hours. I will demonstrate each feature on a live system while the teacher follows along on their own device. The reason I am doing this in-person rather than just giving the teacher a written guide is that the analytics section needs explanation — a teacher needs to understand what a risk score of 0.72 actually means in practice, why inactivity is weighted at 20%, and when they should act on a Level 2 alert versus a Level 3 alert.

We will cover these topics in order:

1. Registering a Teacher account and understanding the role selection screen
2. Creating a course — title, description, and the publish/draft toggle
3. Adding modules and lessons. Walking through how the three-level hierarchy (Course > Module > Lesson) works.
4. Building a quiz — adding questions, entering four answer choices, marking the correct one, setting the pass mark
5. Publishing the course and checking that it appears in the student browse view
6. Reading the analytics dashboard — going through each stat card, the risk distribution donut chart, the student risk table, and the lesson difficulty ranking
7. Understanding the three alert levels: Level 1 (inactive 3+ days), Level 2 (quiz average below 50% with medium/high risk), Level 3 (high risk, inactive 7+ days, progress below 50%)
8. Resolving an alert — editing the pre-written intervention message and logging it
9. Opening the individual Student Detail page and reading the risk history chart

4.3.2 Quick Start Card — for Students

Students do not need a full training session. I will prepare a one-page PDF Quick Start Card that covers the five things a student needs to do on TeachLink. The card uses numbered steps and will include small screenshots. It is short enough to read in under five minutes.

The student Quick Start Card covers:

1. Go to the TeachLink URL, click Register, fill in your name and email, choose the Student role, and create a password
2. After logging in, click My Courses in the sidebar to see courses available to enrol in
3. Click Enrol on any course you want to join
4. Open a lesson, read through the content, and click the lesson title to mark it as visited — your progress percentage will update automatically
5. If a lesson has a quiz attached, click Take Quiz, choose your answers, and click Submit — your score is saved immediately and counts towards your average
6. To see your progress, quiz scores, and how you compare to the class average, click My Analytics in the sidebar

4.3.3 Interactive Demo File — for All Users

Alongside the live system, I built a standalone HTML demo file called `teachlink_demo.html`. It runs entirely in the browser with no server needed — the teacher or student just opens the file and can click through all the views. It has a Teacher/Student toggle in the header to switch between the two perspectives. This is useful for showing the system to someone before they create a real account, or for letting a student explore what the analytics page will look like.

4.3.4 Admin Panel Overview — for System Administrators

Django comes with a built-in admin panel at /admin. Anyone with a superuser account can access it. The admin panel shows every database table as a list view and allows records to be created, edited, or deleted directly. For TeachLink, I have registered all the key models — Users, Courses, Enrollments, Quiz Results, Risk History, Alerts, and Intervention Logs — so everything can be inspected without writing any SQL.

The admin training covers:

- Logging in with the superuser credentials created during installation
- Navigating to the Enrollments list and reading risk scores and alert statuses
- Viewing the RiskHistory table to check that the engine is saving records correctly
- Deleting test data before a real go-live (e.g., removing quiz results created during testing)

4.4 Training Schedule

Session	Who Attends	Method	How Long	When
Teacher onboarding	Educator	Live walkthrough	2–3 hours	1 week before go-live
Student quick start	All students	Quick Start Card + demo	~30 minutes	Go-live day
Admin panel briefing	Administrator	Live demonstration	1 hour	During setup
Post-launch support	All users	Email / in-person Q&A	As needed	Week 1 after launch

4.5 Training Materials to Prepare

Document	Format	Who It Is For
Teacher User Guide	PDF with annotated screenshots	Teachers
Student Quick Start Card	One A4 page, PDF or printed	Students
Interactive Demo File	teachlink_demo.html — opens in browser	All users
Risk Score Reference Card	One-page PDF explaining the formula, thresholds, and alert levels	Teachers
Admin Panel Notes	Short written guide	System administrator

5.0 Software Maintenance Plan

Even after TeachLink is deployed and running correctly, it will need ongoing maintenance. Software does not stay stable on its own — Django and Python release security patches, browsers update and sometimes break things, and users will find edge cases that testing missed. This section describes how I plan to handle all of that.

5.1 Types of Maintenance That Will Be Needed

Type	What It Means	TeachLink-Specific Examples
Corrective	Fixing bugs and errors that appear after deployment	The risk score is not updating after a quiz submission; an alert is generated twice in one day; a student cannot enrol because of a database constraint error
Adaptive	Updating the system to keep working as its environment changes	Upgrading Django from 4.2.7 to a newer version; updating psycopg2 when PostgreSQL releases a new major version; fixing a chart display issue caused by a Chrome update
Perfective	Improving features based on feedback from real use	The teacher finds the student risk table hard to read when there are many students — adding a filter or sort option; a student finds the quiz interface confusing — redesigning the question layout
Preventive	Making changes now to avoid bigger problems later	Adding stricter input validation to the quiz form to prevent empty questions being saved; refactoring the risk engine to make it easier to adjust the weighting formula in future

5.2 Who Is Responsible for What

Role	Responsibilities
Developer (me)	Investigate reported bugs, reproduce them locally, write and test the fix, and deploy it. Also responsible for keeping Django and Python dependencies up to date.
Teacher	Report any unexpected behaviour clearly — describe what they were doing, what they expected, and what actually happened. Confirm when a fix resolves the issue.
System Administrator	Monitor server uptime, run weekly database backups, apply operating system security patches, and restart the application server if it crashes.

5.3 Bug Reporting and Fix Process

When a bug is reported, I will follow this process every time to make sure nothing gets fixed incorrectly or deployed without being tested:

1. The user who found the bug writes down: what page they were on, what they did, what they expected to happen, and what actually happened instead.
2. They send this to me by email or through the shared issue log.
3. I reproduce the bug in my local development environment using the same steps. If I cannot reproduce it, I ask the user for more details.
4. I classify the severity — Critical means the system is unusable; High means a core feature is broken; Medium means a minor feature has an issue; Low means it is cosmetic only.
5. I write the fix in development, test it manually, and — for anything touching the risk engine — run the unit tests to make sure nothing else broke.
6. I deploy the fix to production and tell the user it has been applied.
7. The user confirms the fix works. I mark the issue as resolved in the log.

5.4 Database Backup Procedure

The PostgreSQL database holds everything — all user accounts, course content, quiz results, risk histories, and alert records. Losing it would mean losing all student progress data. Backups must happen regularly.

PostgreSQL includes a command-line tool called `pg_dump` that exports the entire database to a single .sql file. This file can be used to restore the database to that exact state if anything goes wrong.

Backup Type	How Often	Command / Method
Full database export	Every week	<code>pg_dump -U postgres teachlink_db > backup_2026-03-15.sql</code>
Keep last 4 backups	Rolling	Delete backups older than 4 weeks to avoid filling up storage
Pre-deployment snapshot	Before any code update	Always run a manual <code>pg_dump</code> before deploying any change to production

5.5 Keeping Dependencies Up to Date

Django and its ecosystem release security updates regularly. Ignoring these updates means the system could be vulnerable to known exploits. The policy for TeachLink is:

- Security patches (e.g., Django 4.2.7 → 4.2.8 fixing a known vulnerability): apply within two weeks. Check [djangoproject.com/weblog](https://www.djangoproject.com/weblog/) for security announcements.
- Minor updates (e.g., Django 4.2.7 → 4.2.9): test in development first, then apply. These are usually safe but should never be applied directly in production without testing.
- Major version upgrades (e.g., Django 4.x → 5.x): treat carefully. Read the full release notes and check whether any of TeachLink's features use deprecated APIs. Only upgrade after thorough testing.

- Never apply any update directly in production. Always test on the local development environment first.

5.6 Maintenance Log

Every maintenance activity — whether a bug fix, a dependency update, or a server restart — should be recorded. This creates a history that is useful for diagnosing future issues.

Date	Type	What Happened	What Was Done	By

6.0 Change Management Plan

Change management is the process of controlling how updates and modifications to TeachLink get approved and deployed. The reason this matters is that TeachLink is not just a personal project anymore once it is deployed — educators and students will be relying on it. A poorly tested change pushed directly to production could break the risk engine, generate wrong alerts, or prevent students from submitting quizzes. That would affect real people's work.

The plan below is designed to be realistic for a small project with a single developer, while still being structured enough to prevent careless mistakes.

6.1 Categories of Change

Category	What It Covers	Real Examples from TeachLink
Minor	Small changes to the UI, text, or styling that carry no risk of breaking functionality	Rewording an alert message to be clearer; changing a button colour; fixing a typo in a template
Moderate	Changes to existing features that could have a knock-on effect on related parts of the system	Changing the inactivity weight from 20% to 25% in the risk formula; adding a new column to the student risk table; adjusting alert threshold values
Major	New features, database schema changes, or structural modifications	Adding a new analytics module; adding a field to the Enrollment model (requires a migration); changing how the alert deduplication logic works

6.2 The Change Request Process

Every change, regardless of how small it seems, goes through the following steps. The reason for doing this even for minor changes is that what looks like a small text change in a template can sometimes reference a variable that no longer exists, causing a server error.

1. Someone identifies the need for a change — this could be the teacher reporting unexpected behaviour, a student finding something confusing, or me noticing a problem during testing. The change is written down using the Change Request Form in Section 6.5.
2. I classify the change as Minor, Moderate, or Major based on what components it touches.
3. I assess the impact: does it need a database migration? Does it touch the risk engine? Could it affect the alert logic? How long will it take?
4. Minor changes: I can approve and implement these on my own. Moderate and Major changes need to be reviewed by my supervisor before I start work.
5. I implement the change in my local development environment — never directly on the production server.
6. I test the change. For anything touching the risk engine or alert logic, I run the unit tests. For UI changes, I test manually in a browser.
7. Once testing passes and the change is approved, I deploy it to production.
8. If the change affects how any user interacts with the system, I inform the teacher or students before or immediately after deployment.

9. I update any documentation that refers to the changed feature — user guide, this plan, or the SDD.
10. I mark the change request as complete in the change log.

6.3 Roles in the Change Process

Role	What They Do
Change Requestor	Anyone — the teacher, a student, or me — who identifies something that needs changing. They fill in the Change Request Form.
Developer (me)	Classify the change, assess the impact, implement it in development, test it, and deploy it after approval.
Supervisor	Reviews and approves Moderate and Major changes before I start implementing them. Provides a second pair of eyes on anything that could significantly affect how the system works.
Teacher / Students	Notified of any change that affects their workflow. Provide feedback on whether the change fixed the problem or introduced new issues.

6.4 Impact Assessment Checklist

Before any change is approved, I go through the following questions. Each 'Yes' answer means that aspect of the system needs to be included in the testing plan for that change.

- Does the change touch analytics/engine.py or the risk formula? If yes, the unit tests must pass after the change.
- Does the change add, remove, or rename a field on any model? If yes, a database migration is required.
- Does the change affect what is shown on the teacher dashboard or student analytics page? If yes, the teacher needs to be informed.
- Does the change affect the alert generation logic? If yes, test with enrollments at the threshold values to make sure alerts still trigger correctly.
- Does the change affect user authentication or the role check? If yes, test both Teacher and Student accounts after the change.
- Can this change be undone if it breaks something? If not, prepare a rollback plan before deploying.

6.5 Change Request Form

Field	Response
Change Request Number	CR-001
Date	
Requested By	

Category	Minor / Moderate / Major
What needs to change and why	
Which files / components are affected	
Is a database migration required?	Yes / No
How long will implementation take?	
What tests will be run?	
Approved by	
Approval date	
Deployed on	
Outcome / notes	

6.6 Telling Users About Changes

Not every change needs to be communicated — if I fix a backend bug that users never noticed, there is no need to announce it. But if a change affects how something looks or works on their side, I will tell them before it goes live.

- For the teacher: a direct message (WhatsApp or email) at least one day before the change is deployed, explaining simply what is changing and whether anything they regularly do will work differently.
- For students: a note from the teacher on the next time they log in, or a message in whatever group the class uses. Students generally do not need to know about backend changes — only visible UI changes matter to them.
- For major structural changes that affect multiple features: a short written summary shared with all users at least three days before deployment.

6.7 Rollback Plan

If a change causes something to break in production and I cannot fix it quickly, the safest option is to roll back to the previous working version. This is what I will do:

1. Revert the code using git — either `git revert <commit>` (which creates a new commit undoing the change) or `git checkout <previous_commit>` if I need to go back further.
2. If the change included a database migration, I need to reverse it before reverting the code: `python manage.py migrate <app_name> <previous_migration_name>`. For example: `python manage.py migrate analytics 0003_previous`.
3. If the migration cannot be reversed automatically (some schema changes are not reversible by Django), restore the database from the pre-deployment backup made in Step 7 of the installation procedure.
4. Restart the Django server and confirm the system is back to its previous state.

5. Write up what went wrong in the maintenance log so I can understand why the change failed and avoid the same mistake.
6. Fix the issue properly in development, re-test, and re-deploy only when I am confident the fix is correct.

7.0 Conclusion

This implementation plan covers how TeachLink will be deployed and operated from initial installation through to ongoing maintenance. I have tried to write it in a way that is directly connected to how the system actually works — referencing real file names, real formulas, and real steps — rather than using generic project management language.

The four-phase schedule reflects how I actually built the system: starting with the database and authentication layer, then the course and quiz system, then the risk engine and analytics, and finally testing and deployment. The schedule shows that the first three phases are complete and Phase 4 is currently in progress.

The installation guide is specific enough that someone with Python and PostgreSQL knowledge could set up the system from scratch in under an hour. The conversion strategy — Direct Cutover — makes sense for this context because there is no legacy LMS to migrate from; the educators using TeachLink are moving from informal tools, and running two systems in parallel would create confusion rather than reduce risk.

The training plan is practical: a live 2-3 hour walkthrough for the teacher, a one-page quick start card for students, and a standalone demo file that works offline. Training is proportionate to how complex the system actually is to use — students need very little, and teachers mainly need help understanding the analytics side.

The maintenance and change management sections are written for a small, real deployment with one developer. The procedures are structured but not over-engineered — the goal is to prevent careless mistakes while keeping the overhead manageable.

Together, these plans should give a clear picture of how TeachLink moves from a working codebase to a system that independent educators can rely on day to day.

End of Implementation Plan.